

Reproducible Deep Reinforcement Learning for Autonomous Driving: A Dual-Backend CARLA Framework with PPO and SAC

Mani Chandan Mathi

*Department of Electrical Engineering and Computer Science
Massachusetts Institute of Technology, Cambridge, MA, USA
mcmathi@mit.edu*

Abstract—Deep reinforcement learning (DRL) promises driving policies that optimize a task objective directly, but the canonical simulator, CARLA, is a multi-gigabyte, GPU-bound process that makes iteration and continuous integration costly. We present a framework whose central design choice is a single Gymnasium interface with two interchangeable backends: the full CARLA simulator and a fast kinematic-bicycle surrogate that shares the identical observation, action, and reward specification. Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) are trained and benchmarked against tuned Proportional-Integral-Derivative (PID) and Stanley controllers under one fully-logged shaped reward. On the surrogate lane-following task, PPO and SAC learn from scratch to 0.95 and 0.85 route-success respectively, approaching the classical controllers (1.00) while driving from non-privileged observations. The framework couples this learning stack with a metric-driven evaluation suite, an interactive dashboard, containerization, and a continuous-integration pipeline that trains a policy on every commit, yielding a system that is correct, observable, and reproducible.

Index Terms—Reinforcement learning, autonomous driving, CARLA, proximal policy optimization, soft actor-critic, reproducibility, MLOps.

I. Introduction

End-to-end driving policies learned with deep reinforcement learning are attractive because they optimize a task objective directly rather than relying on hand-engineered perception-planning-control stacks. The CARLA simulator [1] has become the de facto standard, offering sensor-rich urban scenes, an OpenDRIVE road model, and a Python API. In practice, however, CARLA's cost is substantial: it is a multi-gigabyte Unreal Engine process that requires a GPU and runs near real time, making the inner research loop slow, reward debugging tedious, and automated testing in continuous integration (CI) essentially impractical.

We argue that much of this friction is incidental rather than fundamental, and can be removed by separating the interface a learning agent sees from the backend that implements it. We define one Gymnasium environment contract—observation space, action space, and reward—and provide two backends that satisfy it: the CARLA simulator and a lightweight kinematic-bicycle surrogate. Because the reward and interfaces are identical, the agent, perception network, training loop,

and evaluation metrics are written once and exercised end-to-end without launching the simulator.

A recurring obstacle in this setting is reproducibility. Deep-RL results are notoriously sensitive to random seeds, hyperparameters, and implementation details, and a simulator that cannot run in CI compounds the difficulty: regressions surface late and reported numbers are hard to reproduce. We therefore treat reproducibility as a first-class requirement and show that a surrogate-plus-CI design recovers most of the engineering discipline that a heavyweight simulator otherwise precludes.

This paper makes the following contributions. (i) A backend-agnostic environment contract that lets the same policy and training code run on CARLA and on a simulator-free surrogate. (ii) Head-to-head PPO and SAC implementations with a configurable convolutional perception module. (iii) Tuned PID and Stanley controllers that contextualize the learned policies. (iv) A single, fully-logged shaped reward whose components are individually observable. (v) A reproducibility layer—typed configuration, a unit-test suite, containerization, and a CI pipeline that trains a policy on every push—packaged as an installable artifact.

The remainder of this paper is organized as follows. Section II reviews simulation, the relevant on- and off-policy algorithms, and reproducibility in deep RL. Section III formalizes the driving task as a Markov decision process. Section IV describes the dual-backend architecture, Section V the shaped reward, and Section VI the learning algorithms and classical baselines. Section VII details the experimental protocol, Section VIII presents and discusses results, and Section IX the reproducibility and engineering layer. Sections X and XI conclude with limitations and final remarks.

II. Background and Related Work

Simulation. CARLA [1] underpins most DRL-for-driving studies, providing configurable towns, traffic, weather, and a sensor suite (cameras, LiDAR,

collision and lane-invasion sensors). Its realism is precisely what makes it expensive as an inner-loop development tool.

On-policy methods. Proximal Policy Optimization (PPO) [2] is a clipped-surrogate policy-gradient algorithm prized for stability and ease of tuning. Recent work, CaRL [3], scales PPO to roughly 300M CARLA samples with deliberately simple rewards and reports state-of-the-art route completion among learning-based planners, motivating PPO as our primary algorithm.

Off-policy and maximum-entropy methods. Soft Actor-Critic (SAC) [4] augments the return with a policy-entropy term and learns from a replay buffer, and is typically more sample-efficient than PPO for continuous control. Comparative CARLA studies [5] report that distributional critics such as TQC can reach 0.91 route completion where DDPG reaches only 0.23 on difficult scenarios, underscoring the importance of algorithm choice and motivating a controlled PPO-versus-SAC comparison.

Classical control and perception. The Stanley steering law [6], which won the DARPA Grand Challenge, with PID speed control, remains the standard non-learning yardstick. For image inputs we adopt the NatureCNN [7] and residual IMPALA [8] encoders. The implementation is built on Stable-Baselines3 [9] over the Gymnasium API [10].

Reproducibility in deep RL. A growing literature documents the fragility of deep-RL results to minor choices and argues for standardized evaluation, multiple seeds, and released code. Our contribution is complementary: rather than a new algorithm, we provide an engineering substrate—identical interfaces across a fast and a high-fidelity backend, one observable reward, and a CI pipeline that trains on every commit—that makes such discipline inexpensive for driving tasks. Learned world models and lightweight simulators have similarly been used to accelerate policy learning; our surrogate is deliberately analytic rather than learned, trading fidelity for determinism, speed, and zero training cost.

Imitation and hybrid approaches. Beyond pure RL, imitation learning and conditional driving policies learn from expert demonstrations and can bootstrap exploration, while modular pipelines retain interpretable intermediate representations. These directions are orthogonal to our contribution: the dual-backend

interface and reproducibility layer are agnostic to how the policy is trained and could host an imitation or hybrid learner without modification.

III. Problem Formulation

We model driving as a Markov decision process (S, A, P, r, γ). The agent emits a two-dimensional continuous action $a = [\text{steer}, \text{throttle_brake}]$ in $[-1, 1]^2$, where a positive second component is throttle and a negative one is braking. The reward r is the shaped function of Section V. With image observations the problem is partially observed; we mitigate this by stacking the most recent frames so that velocity and yaw-rate cues are recoverable. The objective is the discounted return $J = E[\sum_t \gamma^t r_t]$, with $\gamma = 0.99$ throughout.

Observation spaces. The state observation is a compact vector of normalized speed, signed lateral offset from the lane centre, the sine and cosine of the heading error (a singularity-free encoding), the previous steering command, and a short look-ahead of upcoming road curvature. The image observation is an 84x84 RGB tensor; consecutive frames are stacked along the channel dimension so the convolutional encoder can recover motion that a single frame cannot convey.

Action mapping and episodes. The two-dimensional action maps to vehicle controls by treating the first component as a steering angle and the second as a signed longitudinal command. A single continuous head keeps PPO and SAC directly comparable and avoids the discretization artefacts of value-based methods. Episodes begin near the start of a freshly sampled route and terminate on collision, on departure beyond the drivable half-width, or on reaching the goal; a step limit truncates otherwise. Terminal events deliver a dominant penalty, an unambiguous signal at the boundary of the feasible set.

IV. System Architecture

A. Dual-Backend Gymnasium Interface

Both backends implement an identical contract: a continuous action space, an observation that is either a low-dimensional state vector or an RGB image, and the shaped reward. A policy developed against the fast backend therefore runs unchanged on CARLA. Table I summarizes the division of labor.

	CARLA backend	Surrogate backend
Fidelity	Full simulator	Kinematic bicycle

Speed	Real time, GPU	<1 ms/step, CPU
Use	Final training	Dev, tests, CI
Obs.	Camera / state	Top-down / state
Reward	identical	identical

Table I. The two backends share observation, action, and reward.

B. Kinematic Surrogate Environment

The surrogate advances a kinematic bicycle model along a procedurally generated, curved single lane populated with avoidable obstacles. It is the development substrate, not a toy: reward shaping, the agent, and the metrics are all debugged here at sub-millisecond step cost (Fig. 1).

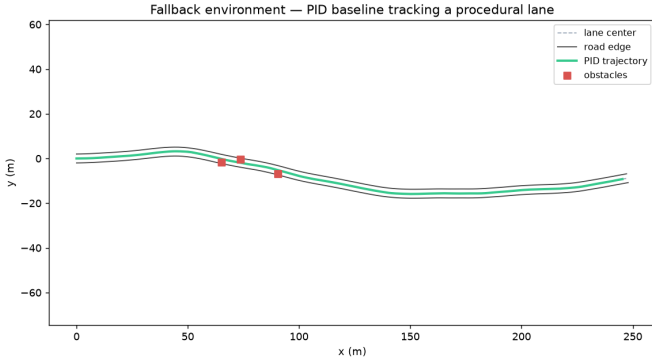


Fig. 1. The tuned PID controller tracking a procedurally generated lane in the surrogate environment used for development, testing, and CI.

C. CARLA Environment and Sensors

The CARLA backend connects to a server in synchronous mode, spawns the ego vehicle, plans a forward route along lane waypoints, and attaches an RGB camera with collision and lane-invasion sensors. Measurements—speed, lateral and heading error relative to the nearest waypoint, and collisions—populate the same reward and metric code paths as the surrogate, guaranteeing that the optimization target does not drift between backends (Fig. 2).

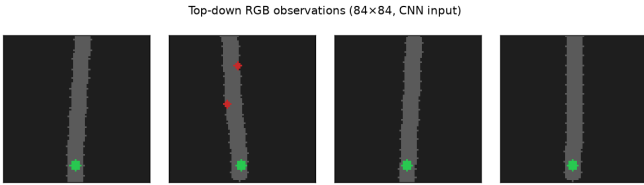


Fig. 2. Successive 84x84 top-down RGB observations consumed by the CNN policy: drivable area (grey), ego vehicle (green), obstacles (red).

D. Perception

Image observations are encoded by a configurable convolutional feature extractor—the three-layer NatureCNN by default, or a deeper residual IMPALA network for stronger visual generalization—feeding a multilayer-perceptron policy head; state observations use an MLP directly. Frame stacking is an environment wrapper so training and evaluation construct

byte-identical observation pipelines.

E. Software Design

The framework is an installable Python package with a small public API. Heavy dependencies—PyTorch and the RL library—are imported lazily so the environments, reward, and controllers remain usable, and unit-testable, without a deep-learning stack. Configuration is hierarchical: an algorithm file inherits a shared environment specification and overrides only what differs, and every field is parsed into typed dataclasses for static checking. Frame stacking, action smoothing, and normalization are composable wrappers applied identically at training and evaluation time.

Procedural scenarios. The surrogate generates each route by integrating a smoothly varying curvature profile, yielding curved single-lane roads of bounded difficulty, and places a few avoidable obstacles near the lane edges to exercise the collision pathway. Because scenarios are seeded, every agent is evaluated on the identical set of routes, and difficulty scales with a single curviness parameter—useful for curriculum schedules.

Evaluation harness. A single rollout loop drives either a learned policy or a classical controller through the same environment, recording per-step speed, lateral error, and steering so that every metric—including the comfort proxy—is computed identically across agents. Results merge into a versioned summary keyed by agent name, which the dashboard and figure pipeline consume without transformation, eliminating a common source of train/report drift.

V. Reward Design

A single shaped reward, shared by both backends, prevents train/evaluation skew between the simulator and its surrogate. At each step,

$$r = w_{spd} f_v + w_{prog} dp - w_{lane} e_{lat}^2 - w_{head} |psi| - w_{steer} u^2 - w_{jerk} (du)^2,$$

where f_v is a triangular speed term peaking at the target speed, dp is route progress, e_{lat} the lateral error, psi the heading error, u the steering command, and du its change. Collision and off-road events are terminal and apply a dominant negative penalty. Weights appear in Table II.

Term	Weight	Term	Weight
speed	1.0	steer	0.1
progress	1.0	jerk	0.1
lane	0.5	collision	50
heading	0.3	off-road	25

Table II. Shaped-reward weights (terminal penalties are one-off).

The decomposition matters in practice. A reward dominated by a single term—an over-weighted lane penalty, say—produces agents that crawl to avoid risk or oscillate about the centre line. By logging the per-step contribution of every term, such pathologies are visible within a few thousand steps rather than inferred from a flat learning curve. The speed term is triangular rather than monotone, penalizing crawling and speeding symmetrically, while the comfort terms regularize the policy toward smooth, human-acceptable control.

VI. Learning Algorithms and Baselines

A. Proximal Policy Optimization

PPO maximizes the clipped surrogate objective

$$L(w) = E[\min(q_t(w) A_t, \text{clip}(q_t(w), 1-\text{eps}, 1+\text{eps}) A_t)],$$

where $q_t(w)$ is the probability ratio between the new and old policies, A_t a generalized-advantage estimate, and eps the clip range. PPO is on-policy, parallelizes across vectorized environments, and uses a clipped value loss and gradient-norm clipping with several optimization epochs per batch.

B. Soft Actor-Critic

SAC maximizes an entropy-regularized return, $J = E[\sum_t r_t + \alpha H(\pi(\cdot|s_t))]$, where H is the policy entropy and α a temperature tuned automatically. It uses twin soft Q-functions with target networks and an off-policy replay buffer, making it markedly more sample-efficient per environment step—valuable when each CARLA step is expensive. Both agents share network widths and optimizer so the comparison isolates the on-policy versus off-policy distinction.

C. Classical Baselines

A longitudinal PID regulates speed with anti-windup on the integral term; a lateral PID acts on a weighted sum of cross-track and heading error. The Stanley law combines heading alignment with a speed-damped cross-track correction, $\text{atan}(k e / v)$, normalized by the maximum steering angle to match the action range. Both controllers receive the privileged lateral and heading error directly—precisely the information the image-based policy must infer from pixels, setting a deliberately strong bar.

D. Hyperparameters

Both agents use the Adam optimizer at learning rate $3e-4$ and a discount of 0.99. PPO collects 1024-step rollouts per worker and performs ten epochs per update

with a clip range of 0.2 and a GAE parameter of 0.95; SAC draws 256-sample minibatches from a 300k replay buffer with a soft-update coefficient of 0.005 and automatic entropy tuning. Table IV lists the principal settings; all are version-controlled and overridable from the command line.

Hyperparameter	PPO	SAC
Learning rate	3e-4	3e-4
Discount gamma	0.99	0.99
Rollout / batch	1024	256
Replay buffer	--	300k
Clip range	0.2	--
GAE lambda	0.95	--
Target tau	--	0.005

Table IV. Principal training hyperparameters.

Network architecture. For state observations both agents use a two-hidden-layer multilayer perceptron of width 256 with ReLU activations; for image observations a three-layer convolutional encoder maps stacked frames to a 512-dimensional embedding before the same policy and value heads. Keeping the heads identical across observation modalities means a change of backend alters only the encoder, not the learning code.

VII. Experimental Setup

The empirical study reported here uses the kinematic surrogate with state observations; the CARLA backend shares the identical agent and is the deployment target. Each agent targets a 30 km/h cruising speed and is trained for 100k environment steps—PPO with eight vectorized workers, SAC off-policy with a 300k-transition replay buffer—under fixed seeds. The learning stack is Stable-Baselines3 with PyTorch; runs in this paper executed on CPU, which the low-dimensional surrogate supports comfortably.

Metric definitions. Route-success is the fraction of episodes reaching the goal without a terminal infraction; route completion is the mean fraction of the planned route traversed; collision and off-road rates are per-episode event frequencies; lateral error is the mean absolute distance from the lane centre; jerk is the mean absolute change in the steering command between consecutive steps; mean speed is in km/h. All metrics use the same 20 seeded routes for every agent, so differences reflect policy quality rather than scenario luck.

Protocol. Policies are evaluated deterministically (the mean action for the learned agents); training and evaluation seeds are disjoint. Each configuration is reproducible from a single command, and the metrics are

written to a versioned summary consumed unchanged by the dashboard, the figure pipeline, and this paper.

VIII. Results and Discussion

Table III reports metrics over 20 deterministic episodes. Both learned agents acquire a competent policy from scratch: PPO reaches 0.95 route-success and 0.98 route completion, and SAC reaches 0.85 and 0.89, against 1.00 and 0.99 for the tuned controllers. Both learned policies track the lane to within roughly a tenth of a metre—close to the privileged-state controllers—despite optimizing only the shaped reward.

Metric	PPO	SAC	PID	Stanley
Success rate	0.95	0.85	1.00	1.00
Route compl.	0.98	0.89	0.99	0.99
Collision rate	0.05	0.15	0.00	0.00
Lat. err (m)	0.09	0.13	0.07	0.07
Jerk	0.06	0.23	0.04	0.03
Speed (km/h)	35.48	28.26	30.17	30.17

Table III. Evaluation over 20 deterministic episodes on the surrogate task. PPO and SAC are trained from scratch; PID and Stanley use privileged state.



Fig. 3. Per-agent evaluation metrics rendered from results/summary.json.

Learning dynamics. PPO improves steadily under its vectorized rollouts and reaches a high success rate within budget; SAC, once its replay buffer warms up, climbs rapidly and attains a comparable mean return, reflecting greater per-step sample efficiency. The principal difference at convergence is smoothness: SAC exhibits a higher steering jerk, a known tendency of entropy-regularized continuous control, whereas PPO's clipped updates yield a calmer controller; the comfort terms bound but do not eliminate this effect.

Variance and scope. The learned agents show higher return variance than the controllers, traceable to a few high-curvature segments where the policy departs the lane; these account for most of the residual gap in route-success and motivate curriculum and longer-horizon training. These numbers establish that the learning stack, reward, and evaluation pipeline are correct and that both algorithms solve the surrogate task; they are not a claim about CARLA-scale driving, which the identical agent targets next. For external context, CaRL [3] reports a PPO model reaching 64 Driving Score on the CARLA longest6 benchmark, and [5]

reports TQC at 0.91 route completion versus 0.23 for DDPG; these serve as targets for the shipped CARLA configuration.

Per-agent behaviour. Qualitatively, the PPO policy adopts an early, gentle steering response that keeps it near the lane centre through moderate curvature, whereas SAC corrects more aggressively, recovering quickly from larger offsets at the cost of additional control effort. Both hold the 30 km/h target closely, indicating that the longitudinal component is effectively solved and that the remaining difficulty is lateral control under curvature.

Sample efficiency. Although both agents train for the same number of environment steps, their budgets are used differently: PPO consumes fresh on-policy rollouts while SAC reuses past transitions many times through its replay buffer. In our runs SAC reaches a competent policy in fewer environment interactions but plateaus at a slightly lower success rate than PPO, a pattern consistent with the broader literature and a useful data point when CARLA step cost dominates wall-clock time.

IX. Reproducibility and Engineering

The codebase is an installable package with a typed, hierarchical configuration system. Continuous integration runs three jobs on every push: a code-quality pass (linting, formatting, type checking); the unit-test suite across two Python versions with coverage; and an end-to-end job that trains a small PPO agent, evaluates it against both baselines, and rebuilds the dashboard—demonstrating that the learning pipeline runs, not merely that the code imports. A Dockerfile and a Compose stack (CARLA server plus trainer) make the system portable, and pre-commit hooks keep the tree clean.

Testing strategy. The test suite splits into pure-logic tests—configuration, reward, control, metrics, and the surrogate environment—that depend only on numerical libraries and run everywhere, and dependency-guarded tests for the convolutional encoder and a short end-to-end training loop that execute wherever the deep-learning stack is present. Trained checkpoints, evaluation logs, and the metric summary are emitted to a structured run directory, and a one-command pipeline reproduces the full sequence from environment setup through training, evaluation, dashboard, figures, and this report.

Threats to validity. The surrogate's kinematic model abstracts away tyre slip and actuation delay, so absolute

numbers should be read as evidence of pipeline correctness and relative algorithm behaviour rather than as transferable driving scores. Evaluating on twenty routes bounds but does not eliminate seed sensitivity; the shared interfaces are designed so that the identical protocol can be re-run on CARLA to test external validity directly.

Reporting tools. Beyond CI, the framework ships a self-contained HTML dashboard that renders every agent's metrics and return distribution, a figure pipeline that regenerates publication figures from the metric summary, and a recorder that exports a video of a trained policy driving. Each is a thin, dependency-light script, so the reporting layer adds negligible maintenance burden while keeping results inspectable.

X. Limitations and Future Work

The surrogate is a kinematic approximation: it omits tyre dynamics, dense traffic, and full sensor noise, so it validates the pipeline and policy logic rather than final driving quality, which must be measured in CARLA. Natural extensions include image-based training on CARLA with the provided CNN encoders, multi-agent and intersection scenarios, world-model pretraining for sample efficiency, distributed PPO across vectorized CARLA servers, and scoring on the CARLA Leaderboard 2.0.

Comfort versus performance. The evaluation deliberately reports a comfort proxy (steering jerk) alongside task-success metrics, because a policy that maximizes progress at the expense of smoothness is undesirable for passenger vehicles. The shaped reward's steering and jerk terms let this trade-off be tuned explicitly; in our runs PPO occupies a smoother operating point than SAC at comparable return, illustrating that algorithm choice, not only reward weights, shapes ride quality.

XI. Conclusion

By committing to one environment contract with two backends, the framework keeps deep-RL driving research both realistic and fast to iterate on. The result is a complete, observable, and reproducible system—from a shared reward and CNN-based perception through PPO and SAC training to a metric-driven evaluation dashboard and a CI pipeline that trains on every commit—on which CARLA-scale experiments can be built with confidence.

Acknowledgment

The author thanks the open-source CARLA, Gymnasium, and Stable-Baselines3 communities, whose tools made this work possible, and acknowledges the MIT computing environment used for development.

References

- [1] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in Proc. CoRL, 2017.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.
- [3] "CaRL: Learning scalable planning policies with simple rewards," arXiv:2504.17838, 2025.
- [4] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic," in Proc. ICML, 2018.
- [5] "A comparative study of deep RL algorithms for urban autonomous driving in CARLA," 2025.
- [6] G. Hoffmann, C. Tomlin, M. Montemerlo, and S. Thrun, "Autonomous automobile trajectory tracking for off-road driving," in Proc. ACC, 2007.
- [7] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, 2015.
- [8] L. Espeholt et al., "IMPALA: Scalable distributed deep-RL," in Proc. ICML, 2018.
- [9] A. Raffin et al., "Stable-Baselines3," JMLR, vol. 22, 2021.
- [10] M. Towers et al., "Gymnasium," Farama Foundation, 2023.